Lezione 3

Argomenti	Algebra Booleana

📘 Lezione 3 – Esercizi su logica binaria e algebra di Boole

👨‍🏫 **Obiettivo del tutorato:** Applicare e consolidare i concetti di **logica binaria**, **algebra di Boole** e **conversioni tra sistemi numerici**, rispettando quanto fatto a lezione. Tutti gli esercizi sono svolgibili in **Python**, senza usare `if`, `for` o `while`. Ogni esercizio include un breve richiamo teorico per facilitare la comprensione.

🔢 Esercizio 1 – Da binario a decimale (4 bit)

📘 **Consegna:**

Leggere i valori (0 o 1) dei quattro bit `b3, b2, b1, b0` e calcolare il valore **decimale** corrispondente.

💡 **Promemoria teorico:**

Il sistema binario è posizionale: ogni cifra rappresenta una **potenza di 2**.

Formula: $b3 \cdot 2^3 + b2 \cdot 2^2 + b1 \cdot 2^1 + b0 \cdot 2^0$

📥 **INPUT:**

```
b3=1, b2=0, b1=1, b0=0
```

📤 **OUTPUT:**

```
10
```

✅ **Soluzione:**

```
b0 = eval(input("b0: "))
b1 = eval(input("b1: "))
b2 = eval(input("b2: "))
b3 = eval(input("b3: "))
```

```
decimale = b3*2**3 + b2*2**2 + b1*2**1 + b0*2**0
print(decimale)
```

🧠 *Questo esercizio aiuta a capire come le potenze di due formano i numeri binari.*

🔁 Esercizio 2 – Da decimale a binario (0–15)

📘 **Consegna:**

Leggere un numero decimale compreso tra 0 e 15 e stampare i 4 bit binari corrispondenti.

💡 **Promemoria teorico:**

La conversione decimale-binario si ottiene tramite **divisioni successive per 2**. In Python, si usano `//` (divisione intera) e `%` (resto). Ogni divisione individua quante volte una potenza di 2 è contenuta nel numero, e il resto determina il bit corrispondente.

📥 **INPUT:**

```
10
```

📤 **OUTPUT:**

```
1 0 1 0
```

✅ **Soluzione:**

```
n = eval(input("Inserisci un numero tra 0 e 15: "))

b3 = n // 8
n = n % 8
b2 = n // 4
n = n % 4
b1 = n // 2
b0 = n % 2
```

```
print(b3, b2, b1, b0)
```

🧠 **Spiegazione logica:**

Il programma calcola la rappresentazione binaria analizzando le potenze di 2 che compongono il numero:

- `b3` indica se il numero contiene almeno un gruppo da 8 (2^3)
- `b2` indica se, tolti gli 8, restano gruppi da 4 (2^2)
- `b1` indica se restano gruppi da 2 (2^1)
- `b0` è il resto finale, 0 o 1 (2^0)

Ogni passo divide il numero e conserva il resto, costruendo così i bit dal più significativo (b3) al meno significativo (b0).

📘 *Esempio: per n = 10 → b3=1, b2=0, b1=1, b0=0 → 1010_2.*

⚙️ Esercizio 3 – Somma logica (OR)

📘 **Consegna:**

Leggere due valori binari `a` e `b` (0 o 1) e calcolare la **somma logica** (operatore OR).

💡 **Promemoria teorico:**

`a + b` è 1 se **almeno uno** dei due valori è 1. È l'equivalente logico del "**oppure**".

📥 **INPUT:**

```
a=0, b=1
```

📤 **OUTPUT:**

```
a + b = 1
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
print("a + b =", int(a or b))
```

🧠 *Prima forma logica elementare: mostra il comportamento dell'OR tra due bit.*

🧮 Esercizio 4 – Prodotto logico (AND)

📘 **Consegna:**

Leggere due valori binari `a` e `b` e calcolare il **prodotto logico** (operatore AND).

💡 **Promemoria teorico:**

`a · b` restituisce 1 solo se **entrambi** sono 1. È l'equivalente logico di "**e contemporaneamente**".

📩 **INPUT:**

```
a=1, b=0
```

📤 **OUTPUT:**

```
a · b = 0
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
print("a · b =", int(a and b))
```

🧠 *Mostra come il prodotto logico imponga che entrambe le condizioni siano vere.*

🧩 Esercizio 5 – Negazione logica (NOT)

📘 **Consegna:**

Leggere un valore binario `x` e calcolare il suo **complemento logico** (`x'`).

💡 **Promemoria teorico:**

La negazione logica inverte il valore del bit: 0 → 1, 1 → 0. È l'operazione di **complemento**.

📩 **INPUT:**

```
x=1
```

📩 **OUTPUT:**

```
x' = 0
```

✅ **Soluzione:**

```
x = eval(input("x: "))
print("x' =", int(not x))
```

🧠 *Utile per introdurre il concetto di complemento e la logica inversa.*

🧠 Esercizio 6 – Legge di De Morgan (2 variabili)

📘 **Consegna:**

Verificare la prima legge di De Morgan:

`(a + b)' = a' · b'`

💡 **Promemoria teorico:**

Le **leggi di De Morgan** mostrano la relazione tra negazione, somma e prodotto logico.

Servono per semplificare espressioni booleane complesse.

📩 **INPUT:**

```
a=1, b=0
```

📩 **OUTPUT:**

```
(a + b)' = 0
a' · b' = 0
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
```

```
print("(a + b)' =", int(not (a or b)))
print("a' · b' =", int((not a) and (not b)))
```

🧠 *Serve per comprendere la simmetria tra OR e AND sotto negazione.*

🧠 Esercizio 7 – Legge di De Morgan (3 variabili)

📘 **Consegna:**

Verificare la generalizzazione della legge di De Morgan:

`(a + b + c)' = a' · b' · c'`

💡 **Promemoria teorico:**

Quando la negazione coinvolge più variabili in somma, diventa un prodotto di negazioni:

Negazione di OR → AND di NOT.

📩 **INPUT:**

```
a=0, b=1, c=1
```

📤 **OUTPUT:**

```
(a + b + c)' = 0
a' · b' · c' = 0
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
c = eval(input("c: "))
print("(a + b + c)' =", int(not (a or b or c)))
print("a' · b' · c' =", int((not a) and (not b) and (not c)))
```

🧠 *Mostra come la regola si estende a qualsiasi numero di variabili.*

🧮 Esercizio 8 – Legge distributiva

📘 **Consegna:**

Verificare la legge distributiva:

`a · (b + c) = a·b + a·c`

💡 **Promemoria teorico:**

La legge distributiva collega **prodotto e somma logica**, come in aritmetica:

`a(b+c) = ab + ac`. È utile per ridurre o espandere espressioni logiche.

📥 **INPUT:**

```
a=1, b=0, c=1
```

📤 **OUTPUT:**

```
a · (b + c) = 1
a·b + a·c = 1
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
c = eval(input("c: "))
print("a · (b + c) =", int(a and (b or c)))
print("a·b + a·c =", int((a and b) or (a and c)))
```

🧠 *Mostra la simmetria tra logica booleana e algebra tradizionale.*

🧮 Esercizio 9 – Legge di assorbimento

📘 **Consegna:**

Verificare le due forme della **legge di assorbimento**:

1. `x + x·y = x`
2. `x · (x + y) = x`

💡 **Promemoria teorico:**

Questa legge dimostra che un termine già presente in una somma o in un prodotto **assorbe** quello più complesso.

📥 **INPUT:**

```
x=1, y=0
```

📩 **OUTPUT:**

```
x + x·y = 1
x · (x + y) = 1
```

✅ **Soluzione:**

```
x = eval(input("x: "))
y = eval(input("y: "))
print("x + x·y =", int(x or (x and y)))
print("x · (x + y) =", int(x and (x or y)))
```

🧠 *Importante per capire come semplificare le espressioni ridondanti.*

🔣 Esercizio 11 – Espressione booleana complessa

📘 **Consegna:**

Calcolare il valore della funzione:

`f(a,b,c) = (a' + c) · a · c' + b'`

💡 **Promemoria teorico:**

Le espressioni complesse combinano più operatori. È utile eseguirle passo passo, ricordando le **precedenze logiche**: NOT → AND → OR.

📩 **INPUT:**

```
a=1, b=0, c=1
```

📩 **OUTPUT:**

```
f(a,b,c) = 1
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
```

```
c = eval(input("c: "))
f = int((((not a) or c) and a and (not c)) or (not b))
print("f(a,b,c) =", f)
```

🧠 *Esercizio di sintesi: combina OR, AND e NOT in un'unica formula.*

🧮 Esercizio 12 – Espressione alternativa

📘 **Consegna:**

Calcolare la funzione alternativa:

`g(a,b,c) = (a' · b) + a + c`

💡 **Promemoria teorico:**

È una funzione logica più semplice di `f`, utile per osservare differenze strutturali tra formule simili.

📩 **INPUT:**

```
a=1, b=0, c=1
```

📤 **OUTPUT:**

```
g(a,b,c) = 1
```

✅ **Soluzione:**

```
a = eval(input("a: "))
b = eval(input("b: "))
c = eval(input("c: "))
g = int(((not a) and b) or a or c)
print("g(a,b,c) =", g)
```

🧠 *Confronta questa formula con la precedente per riconoscere espressioni equivalenti.*

📅 **Data:** 13 ottobre 2025

👨‍🏫 **Tutor:** Francesco Caligiuri

💻 **Tema:** Logica binaria e algebra di Boole – esercizi in Python

🎥 *Versione per la proiezione in aula (con spiegazioni, input/output e soluzioni)*